Assignment No: 9

Title: Implement auto-scaling for your AI app on Kubernetes and monitor its health and performance metrics.

Steps:

Phase 1: Application & Dockerization

1. Create the AI Application (app.py)

This Flask API includes a computational loop to simulate AI inference spikes, which is necessary to trigger the auto-scaler.

```
None

from flask import Flask, jsonify
import time import math app

= Flask(__name__)


@app.route('/predict', methods=['GET']) def predict():
                                        # Simulate AI workload to trigger CPU scaling [cite:
7] start_time = time.time() for i in
        range(1000000):
                math.sqrt(i)


        return jsonify({
                "status": "success",
                        "prediction": "Model output placeholder",
                "compute_time": f"{time.time() - start_time:.4f}s"
        })
if __name__ == '__main__':
        app.run(host='0.0.0.0', port=5000)
```

2. Create the Dockerfile (Dockerfile)

```
None

FROM python:3.9-slim
WORKDIR /app
```

```
RUN pip install flask
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

3. Build the Image

Run these in your terminal to build the image directly inside Minikube:

```
None
eval $(minikube docker-env) docker build -t ai
app:latest .
```

Phase 2: Kubernetes Orchestration

4. Deployment YAML (deployment.yaml)
You **must** include resource requests so the Horizontal Pod Autoscaler (HPA) can calculate CPU usage.

YAML

```
None
apiVersion: apps/v1 kind:
Deployment metadata:
    name: ai-app spec:
     replicas: 1
    selector:
        matchLabels:
            app: ai-app template:
        metadata:
            labels:
                app: ai-app
        spec: containers:
            - name: ai-container image: ai-app:latest
                imagePullPolicy: IfNotPresent resources:
                    requests:
```

[cite_start]cpu: "100m" # Required for HPA [cite: 36] memory: "128Mi"
limits:
cpu: "500m" memory: "256Mi"

5. Service YAML (service.yaml)

YAML

```
None

apiVersion: v1 kind: Service metadata: name: ai-service
spec: type: NodePort
  selector:
    app: ai-app
  ports:
    - protocol: TCP port: 80
      targetPort: 5000 nodePort: 30001
```

Apply these files:

```
None

kubectl apply -f deployment.yaml kubectl apply -f service.yaml
```

Phase 3: Auto-Scaling & Load Testing

6. Enable HPA

Ensure the Metrics Server is active and set your scaling rules:

```
None

minikube addons enable metrics-server kubectl autoscale deployment ai-app --cpu-percent=50
```

```
--min=1 --max=5
```

7. Generate Load & Monitor

In one terminal, start the load:

```
None
URL=$(minikube service ai-service --url) while true; do curl -s
$URL/predict; done
```

In another terminal, watch the scaling happen:

```
None
kubectl get hpa -w
```

Phase 4: Monitoring Stack

8. Install Prometheus & Grafana

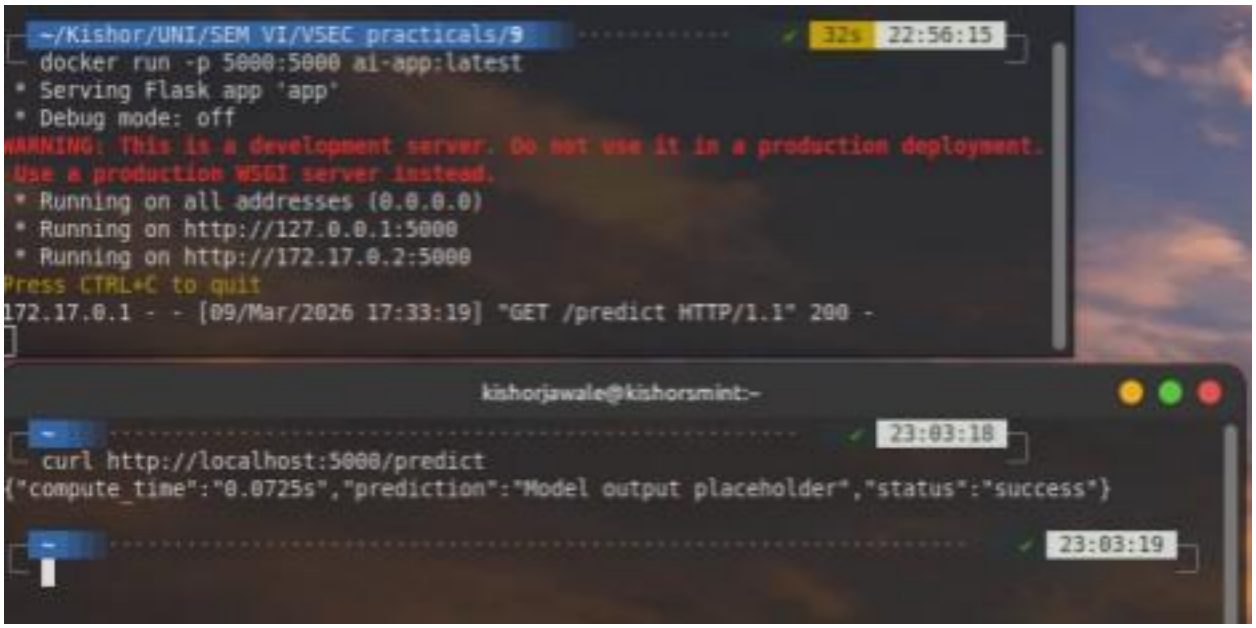
```
None
helm repo add prometheus-community https://prometheus
community.github.io/helm-charts helm repo update helm install
monitoring
prometheus-community/kube-prometheus-stack
```

9. Access Dashboards

```
None
# Get Admin Password
kubectl get secret monitoring-grafana -o
jsonpath="{.data.admin-password}" | base64 --decode ;
echo
# Access Grafana kubectl port-forward svc/monitoring-grafana 3000:80
```

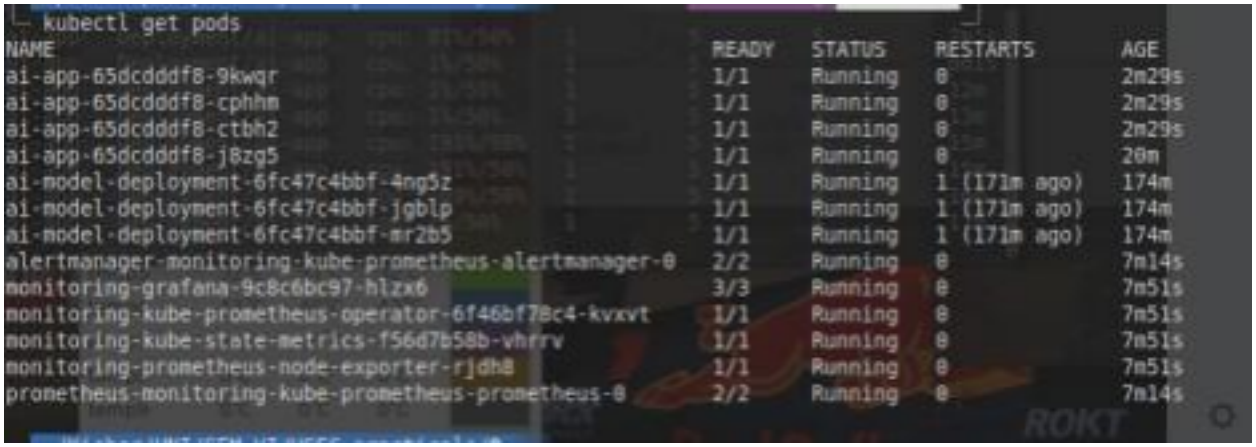
Open http://localhost:3000 to monitor CPU, memory, and pod health.

Output:






CPU Usage
CPU Throttling
CPU Quota
Memory Usage (WSS)


kubectl get pods
NAME READY STATUS RESTARTS AGE
ai-app-65dcdddf8-9kwqr 1/1 Running 0 2m29s
ai-app-65dcdddf8-cphhm 1/1 Running 0 2m29s
ai-app-65dcdddf8-ctbh2 1/1 Running 0 2m29s
ai-app-65dcdddf8-j8zg5 1/1 Running 0 20m
ai-model-deployment-6fc47c4bbf-4ng5z 1/1 Running 1 (171m ago) 174m
ai-model-deployment-6fc47c4bbf-jgblp 1/1 Running 1 (171m ago) 174m
ai-model-deployment-6fc47c4bbf-mr2b5 1/1 Running 1 (171m ago) 174m
alertmanager-monitoring-kube-prometheus-alertmanager-0 2/2 Running 0 7m14s
monitoring-grafana-9c8c6bc97-hlzx6 3/3 Running 0 7m51s
monitoring-kube-prometheus-operator-6f46bf78c4-kvxvt 1/1 Running 0 7m51s
monitoring-kube-state-metrics-f56d7b58b-vhrrv 1/1 Running 0 7m51s
monitoring-prometheus-node-exporter-rjdh8 1/1 Running 0 7m51s
prometheus-monitoring-kube-prometheus-prometheus-0 2/2 Running 0 7m14s